

• 考试的任务

• 学生完成的任务

- 社保资料提供，考勤
- 理论考试的云平台复习（3个工作日之内发放账号和密码），自己去刷8套试题**考试通过率相关**，70%左右可以在云平台刷到

- 理论题题型

二、理论知识考试方案（考试时间 90min）

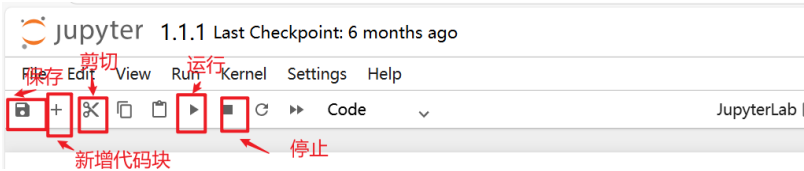
| 题型  | 考试方式 | 认定题量 | 分值/（分/题） | 配分/分 |
|-----|------|------|----------|------|
| 判断题 | 闭卷机考 | 40   | 0.5      | 20   |
| 单选题 |      | 140  | 0.5      | 70   |
| 多选题 |      | 10   | 1        | 10   |
| 小计  | —    | 190  |          | 100  |

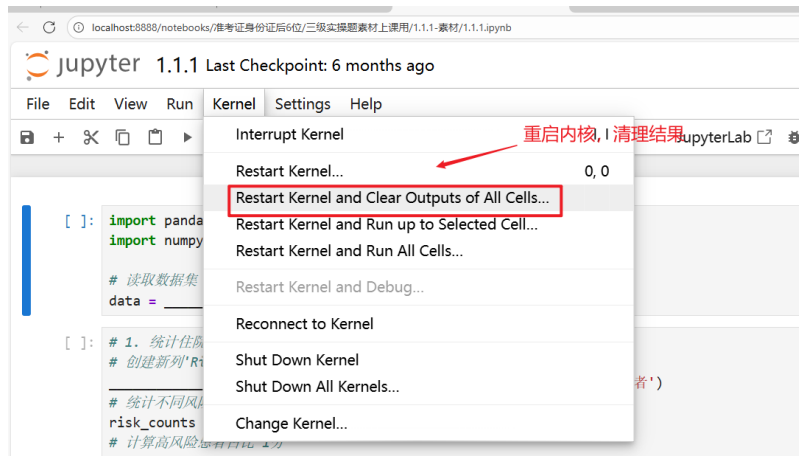
- 云平台的人工智能，python的课程의播放
- 实操题中的文字题( 1.2.X, 3.1.X的部分， 2.1.X， 2.2.X, 3.2.X 中的小问题)

• 一起完成的任务

- 实操题考试环境的搭建(考场环境)

- win11
  - py launcher 勾上
  - python add to path 勾上
  - 一直下一步，下一步，直至安装完成
- office /WPS
- python环境
  - python 主程序 -step1
  - jupyter notebook (人机编程交互的工具，编程体验感更加友好) -step2





- 工具包-step2
- 考试素材下载
  - 三级实操题素材上课用（整理版的实操题库）
  - 人工智能官方手册
    - 理论题和实操题的完整题库（内容多，练习起来不方便）

|                       |                 |                 |            |
|-----------------------|-----------------|-----------------|------------|
| 考核平台演示_人工智能训练师        | 2025/7/13 18:59 | 文件夹             |            |
| 指导手册（人工智能训练师 三级）      | 2025/7/13 18:59 | 文件夹             |            |
| 指导手册（人工智能训练师 四级）      | 2025/7/13 18:59 | 文件夹             |            |
| 指导手册（人工智能训练师 三级）.zip  | 2025/7/18 22:41 | 压缩(zipped)文件... | 258,668 KB |
| 指导手册-人工智能训练师_4级3级.zip | 2025/7/13 18:04 | 压缩(zipped)文件... | 342,552 KB |

- 平时练习的开始步骤
  - 桌面建立考生文件夹
  - 三级实操题素材上课用 放入文件夹
  - 资源管理器左侧点击桌面，上方地址栏输入cmd，然后回车
  - 启动jupyter notebook
    - 1 双击桌面启动bat 脚本（考场上也有这脚本）
    - 2. 命令行启动方式- pylauncher : py -m jupyter notebook
    - 3. 命令行启动方式- add to path : jupyter notebook
    - 意外情况
      - 黑屏不能关闭
      - 如果浏览器跳不出来，地址手动复制到浏览器

http://localhost:8888/?token=923b2724e29a8a115f8aa5da865a7a2f682c9b3fab743f6e  
http://127.0.0.1:8888/?token=923b2724e29a8a115f8aa5da865a7a2f682c9b3fab743f6e

- 导航到桌面考试文件
  -



- 缺库的安装
  - 可以在命令行，也可以在jupyter notebook中安装
  - `pip install 库的名称`
  - 如果网速慢，用国内镜像
    - `pip install -i https://pypi.tuna.tsinghua.edu.cn/simple 库的名称`
- 上传答案
  - 截图-windows 的截图工具，不要用快捷方式
  - 建议搜索截图工具
  - 截图后保存到本地，注意考试的要求上传扩展名是jpg
  - 截图是截取答案部分，不需要代码
  - 上传ipynb，上传之前需要按保存键
  - 其它的上传文件按照考试系统右侧的要求和框来
- 考试流程和重要事项
  - 代码题，100%公布，闭卷考试
  - 代码题，只做填空题（不增加代码和修改代码）
  - 实操题没有通过原因分析
    - 代码题目不熟练
    - 操作时间没有控制好，考试时间来不及
- 实操题
  - 考题类型
    -

| 考核项目表  |      |            |          |      |      |          |    |      |      |
|--------|------|------------|----------|------|------|----------|----|------|------|
| 职业（工种） |      | 人工智能训练师    |          |      | 等级   | 三级       |    |      |      |
| 职业代码   |      | 4-04-05-05 |          |      |      |          |    |      |      |
| 序号     | 项目名称 | 单元编号       | 单元内容     | 考核方式 | 选考方法 | 考核时间（分钟） | 配分 | 题库题量 | 考核题量 |
| 1      | 业务分析 | 1          | 业务流程设计   | 操作   | 抽一   | 30       | 25 | 5    | 1    |
|        |      | 2          | 业务模块效果优化 | 操作   |      | 30       | 25 | 5    |      |
| 2      | 智能训练 | 1          | 数据处理规范制定 | 操作   | 必考   | 20       | 15 | 5    | 1    |
|        |      | 2          | 算法测试     | 操作   | 必考   | 20       | 20 | 5    | 1    |

2

人工智能训练师（三级）认定方案

|    |        |   |           |    |    |     |     |    |   |
|----|--------|---|-----------|----|----|-----|-----|----|---|
| 3  | 智能系统设计 | 1 | 智能系统监控和优化 | 操作 | 必考 | 20  | 15  | 5  | 1 |
|    |        | 2 | 人机交互流程设计  | 操作 | 必考 | 20  | 20  | 5  | 1 |
| 4  | 培训与指导  | 1 | 培训        | 操作 | 抽一 | 10  | 5   | 5  | 1 |
|    |        | 2 | 指导        | 操作 |    | 10  | 5   | 5  |   |
| 合计 |        |   |           |    |    | 120 | 100 | 40 | 6 |

- 第一单元
  - 每个单元分成两个模块 1.1X 和1.2.X （抽一题，总计25分）
 

1.1.1-素材

1.1.2-素材

1.1.3-素材

1.1.4-素材

1.1.5-素材

1.2.1-素材

1.2.2-素材

1.2.3-素材

1.2.4-素材

1.2.5-素材
  - 第一单元总共25分
  - 两个模块总共抽议题
  - 1.1.X 是代码题
  - 1.2.X 是文字题
- 第二单元
  - 代码题必考
  - 两个模块都要抽一题 15分+20分
- 第三单元
  - 3.1.X 是excel 数据透视表分析题
  - 3.2.X 是代码题必考
  - 两个模块都要抽一题 15分+20分

- 第四单元
  - 文字题，两个模块抽一题
  - 总共5分

- 代码题怎么解答

- 1. 看备注
- 2. 看考试的试卷
- 3. 看代码的结构

- 1.1.1

- 工具包的概念

```
[ ]: import pandas as pd
import numpy as np
# 读取数据集 1分
data = _____
```

import 后面就是工具包，这些就是其数据科学家制作的工具包，使得数据分析可以通过简短的自然语言来完成

- 写代码，输入法要英文的

```
import numpy as np
# 读取数据集 1分
data = pd.read_csv('1')
```

1. 双击横杠，退格，去除横杠  
2. pd.read\_csv，连个括号，两个引号，光标到引号当中

```
[ ]: import numpy as np
# 读取数据集 1分
data = pd.read_csv('pa')
# 1. 统计住院天数
# 创建新列'RiskLevel'
# 统计不同风险等级的患者数量 2分
risk_counts = data
```

Tab 键代码提示功能考试被禁用  
文件名可以打  
tab 键连续按，切换对象

| D        | E           | F              | G    | H | I |
|----------|-------------|----------------|------|---|---|
| loadPres | Cholesterol | DaysInHospital | risk |   |   |
| 150      | 211         | 2              | 低    |   |   |
| 118      | 243         | 8              | 高    |   |   |
| 114      | 177         | 8              | 高    |   |   |
| 154      | 237         | 8              | 高    |   |   |
| 120      | 149         | 20             | 高    |   |   |
| 120      | 238         | 2              |      |   |   |
| 107      | 213         | 5              |      |   |   |
| 174      | 203         | 10             |      |   |   |
| 116      | 134         | 6              |      |   |   |
| 178      | 267         | 5              |      |   |   |
| 139      | 276         | 8              |      |   |   |
| 169      | 144         | 5              |      |   |   |
| 107      | 206         | 8              |      |   |   |
| 136      | 101         | 2              |      |   |   |
| 130      | 100         | 15             |      |   |   |

- 构造列

- 表['字段名']
- 如果内存中已经有了这个列，就会更新，没有就新建

```

import pandas as pd
import numpy as np

# 读取数据集 1分
data = pd.read_csv('patient_data.csv')

[ ]: # 1. 统计住院天数超过7天的患者数量及其占比
# 创建新列'RiskLevel', 根据住院天数判断风险等级 3分
data['RiskLevel'] = np.where(data['DaysInHospital'] > 7, '高风险患者', '低风险患者')
# 统计不同风险等级的患者数量 2分
risk_counts = data.groupby('RiskLevel').count()
# 计算高风险患者占比 1分
high_risk_ratio = risk_counts['高风险患者'] / risk_counts.sum()
# 计算低风险患者占比 1分
low_risk_ratio = risk_counts['低风险患者'] / risk_counts.sum()

```

```

data = pd.read_csv('patient_data.csv')

[ ]: # 1. 统计住院天数超过7天的患者数量及其占比
# 创建新列'RiskLevel', 根据住院天数判断风险等级 3分
data['RiskLevel'] = np.where(data['DaysInHospital'] > 7, '高风险患者', '低风险患者')
# 统计不同风险等级的患者数量 2分
risk_counts = data.groupby('RiskLevel').count()
# 计算高风险患者占比 1分
high_risk_ratio = risk_counts['高风险患者'] / risk_counts.sum()
# 计算低风险患者占比 1分
low_risk_ratio = risk_counts['低风险患者'] / risk_counts.sum()
# 输出结果

```

```

[2]: # 1. 统计住院天数超过7天的患者数量及其占比
# 创建新列'RiskLevel', 根据住院天数判断风险等级 3分
data['RiskLevel'] = np.where(data['DaysInHospital'] > 7, '高风险患者', '低风险患者')
# 统计不同风险等级的患者数量 2分
risk_counts = data['RiskLevel'].value_counts()
# 计算高风险患者占比 1分
high_risk_ratio = risk_counts['高风险患者'] / risk_counts.sum()
# 计算低风险患者占比 1分
low_risk_ratio = risk_counts['低风险患者'] / risk_counts.sum()

```

```

# 1. 统计住院天数超过7天的患者数量及其占比
# 创建新列'RiskLevel', 根据住院天数判断风险等级 3分
data['RiskLevel'] = np.where(data['DaysInHospital'] > 7, '高风险患者', '低风险患者')
# 统计不同风险等级的患者数量 2分
risk_counts = data['RiskLevel'].value_counts()

```

```

bmi_labels = ['偏瘦', '正常', '超重', '肥胖']
# 根据BMI值划分指定区间 4分
data['BMIRange'] = pd.cut(data['BMI'], bins=4, labels=bmi_labels, right=False) # 使
# 计算每个BMI区间中高风险患者的比例 2分
bmi_risk_rate = data.groupby('BMIRange')['RiskLevel'].apply(lambda x: (x == '高风险患者').sum() / len(x))
# 统计每个BMI区间的患者数量 1分
bmi_patient_count = data.groupby('BMIRange').count()

# 输出结果
print("BMI区间中高风险患者的比例和患者数:")

```

- 分组的概念
  - 基本案例
  - 本班级按照性别分组，统计平均年龄
  - 分组的三要素

- 分组的对象-性别
- 聚合的对象-年龄
- 聚合的方式-平均
- 分组的代码框架
  - 表.groupby('分组字段名称')['聚合字段名称'].聚合方式()

### 1.1.2

```
File Edit View Run Kernel Settings Help Not Trusted
+ - [ ] Code JupyterLab Python 3 (ipykernel)
data['Value'].fillna(inplace=True)
# 保存清洗后的数据
# 删除用于标记异常值的列，并将清洗后的数据保存到新的CSV文件中 4分
cleaned_data = data.drop('is_abnormal', axis=1)
cleaned_data.to_csv('cleaned_sensor_data.csv', index=False)
print("数据清洗完成，已保存为 'cleaned_sensor_data.csv'")
```

[ ]:

data.columns 这个是属性，没有括号

```
[ ]:
# 2. 按位置统计温度和湿度数据
# 筛选出温度和湿度数据，然后按位置和传感器类型分组，计算每个组的平均值 2分
location_stats = data[data['SensorType'].isin(['Temperature', 'Humidity'])].groupby(['Location', 'SensorType'])['Value'].mean()
# 输出结果
print("每个位置的温度和湿度数据平均值:")
print(location_stats)
```

```
[ ]:
# 3. 数据清洗和异常值处理
# 标记异常值 3分
data['is_abnormal'] = np.where(
    ((data['Temperature'] < -10) | (data['Temperature'] > 50)) |
    ((data['Humidity'] < 0) | (data['Humidity'] > 100)),
    True, False
)
# 输出异常值数量 2分
print("异常值数量:", data['is_abnormal'].sum())
```

主页 工具 文档 2/3 拖拽至上传

(2) 按位置统计温度和湿度数据:

通过补全并运行 Python 代码 (1.1.2.ipynb) 统计每个位置的温度和湿度传感器数据的平均值。将上述统计结果截图以 JPG 的格式保存，命名为“1.1.2-2”。

(3) 数据清洗和异常值处理:

通过补全并运行 Python 代码 (1.1.2.ipynb) 对数据进行清洗，处理异常值。具体要求如下:

- 将明显异常的温度 (< -10 或 > 50) 和湿度 (< 0 或 > 100) 数据进行标记并统计。
- 对缺失值使用前面数据的值 (如果前面值没有采用后面数据的值) 进行填补。
- 将清洗后的数据保存为新文件 cleaned\_sensor\_data.csv。

所有结果文件储存在桌面新建的考生文件夹中，文件夹命名为“准考证号+身份证号后六位”。

### # 3. 数据清洗和异常值处理

# 标记异常值 3分

```
data['is_abnormal'] = np.where(
    ((data['SensorType'] == 'Temperature') & ((data['Value'] < -10) | (data['Value'] >
    ((data['SensorType'] == 'Humidity') & ((data['Value'] < 0) | (data['Value'] > 100))
    True, False
)
```

小明=100万，小明拿了100万  
小明==100万，小明就是行走的100万

| A        | B         | C            | D                  | E        | F |
|----------|-----------|--------------|--------------------|----------|---|
| SensorID | Timestamp | SensorType   | Value              | Location |   |
| 1        | #####     | Light        | 79.63217053691564  | Field3   |   |
| 2        | #####     | Temperature  | 0.6450259431536542 | Field1   |   |
| 3        | #####     | SoilPH       | 94.84965180581636  | Field1   |   |
| 4        | #####     | SoilPH       | 76.63760609271043  | Field1   |   |
| 5        | #####     | SoilPH       |                    | Field4   |   |
| 6        | #####     | Humidity     |                    | Field4   |   |
| 7        | #####     | SoilPH       | 14.948116452139338 | Field1   |   |
| 8        | #####     | SoilMoisture | 80.79195734539273  | Field2   |   |
| 9        | #####     | Light        |                    | Field1   |   |
| 10       | #####     | Temperature  |                    | Field4   |   |
| 11       | #####     | Temperature  | 54.69525633430117  | Field2   |   |
| 12       | #####     | Light        | 38.106035263692526 | Field2   |   |

临近值的插补

填充的方向

```
)
# 输出异常值数量 2分
print("异常值数量:", data['is_abnormal'].sum())
# 填补缺失值
# 使用前向填充和后向填充的方法填补缺失值 4分
data['Value'].fillna(inplace=True)
data['Value'].fillna(inplace=True)
# 保存清洗后的数据
data.fillna(
# 删除用于标记异常的列
cleaned_data = data.drop(columns=['is_abnormal'])
print("数据清洗完成")
异常值数量: 1011
```

shift + tab 查看参数怎么填写  
1. 先要运行一下，报错不用管  
2. shift + tab 查找需要的参数和帮助

```
# 使用前向填充和后向填充的方法填补缺失值 4分
data['Value'].fillna(method='ffill', inplace=True)
data['Value'].fillna(method='bfill', inplace=True)
# 保存清洗后的数据
# 删除用于标记异常的列，并将清洗后的数据保存到新的CSV文件中 4分
cleaned_data = data.drop(columns=['is_abnormal'])
cleaned_data.to_csv('cleaned_sensor_data.csv', index=False)
print("数据清洗完成，已保存为 'cleaned_sensor_data.csv'")
```

cleaned\_data  
出来后，data 就不要  
出现了，易错点

```
[ ]:
# 1. 数据完整性审核
missing_values = data.isnull().sum()
duplicate_values = data.duplicated().sum()
# 输出结果
print("缺失值统计:")
```

括号都不要忘记

# 数据缺失值统计 2分  
# 数据重复值统计 2分



```
print("数据清洗完成, 已保存为 'cleaned_credit_data.csv'")
```

```
[4]:
```

先运行一下, 然后data.duplicat\*?  
铜鼓通配符, 来查找想要的函数

```
data.dupli*}]
```

```
d:\program\python\lib\site-packages\IPython\utils\wildcard.py:70: FutureWarning: _AXI
MES has been deprecated.
  ns[key] = getattr(obj, key)
d:\program\python\lib\site-packages\IPython\utils\wildcard.py:70: FutureWarning: _AXI
MBERS has been deprecated.
  ns[key] = getattr(obj, key)
data.duplicated
```

```
[ ]:
```

#### 1.1.1.4

- 注意包的名称是否是缩写

```
•[1]:
```

```
import pandas
import numpy as np
import matplotlib.pyplot as plt

# 1. 数据收集
# 从本地文件中读取数据 2分
data = pandas.read_csv('user_behavior_data.csv')
print("数据收集完成, 已加载到DataFrame中")

# 打印数据的前5条记录 2分
print(data.head())
```

```
lata = data.dropna()

# 数据类型转换
lata['Age'] = data['Age'].astype(int) # Age数据类型转换为int 2分
lata = lata.astype(float) # PurchaseAmount数据类型转换为float 2分
lata = lata.astype(int) # ReviewScore数据类型转换为int 2分

# 处理异常值 2分
lata = lata[(lata['Age'] > 18, 70)) &
            (data['PurchaseAmount'] > 0) &
```

文本: 1+1=11  
数值类型: 1=1=2

并将数据加载到 DataFrame 中。打印前 5 条数据。

(2) 数据清洗与预处理:

通过运行 Python 代码 (1.1.4.ipynb) 对数据进行清洗和预处理, 具体要求如下:

- 处理缺失值: 对缺失值进行填充或删除。
- 数据类型转换: 确保每个字段的数据类型正确。
- 处理异常值: 删除不合理的年龄、购买金额和评价评分。
- 数据标准化: 对购买金额和评价评分进行标准化处理。

清洗后的数据保存为新文件 cleaned\_user\_behavior\_data.csv

(3) 数据统计:

通过运行 Python 代码 (1.1.4.ipynb), 完成以下数据统计任务:

- 统计每个购买类别的用户数。

```
# 数据类型转换
data['Age'] = data['Age'].astype(int) # Age数据类型转换为int 2分
data['PurchaseAmount'] = data['PurchaseAmount'].astype(float) # PurchaseAmount数据类型
data['ReviewScore'] = data['ReviewScore'].astype(int) # ReviewScore数据类型转换为int

# 处理异常值 2分
data = data[(data['Age'] > 18, 70)) &
            (data['PurchaseAmount'] > 0) &
            (data['ReviewScore'] > 1, 5)]

# 数据标准化
data['PurchaseAmount'] = (data['PurchaseAmount'] -
```

按照顺序——对应

- 数据标准化

- z-score

- 去除量纲
- (字段-字段的平均值)/字段的标准差

### 1.1.5

```
# 数据类型转换
data['Age'] = data['Age'].astype(int) #Age数据类型转换为int 1分
data['Speed'] = data['Speed'].astype(float) #Speed数据类型转换为float 1分
data['TravelDistance'] = data['TravelDistance'].astype(float) #TravelDistance数据类型转换为float
data['TravelTime'] = data['TravelTime'].astype(float) #TravelTime数据类型转换为float

# 处理异常值 2分
data = data[(data['Age'] > 18) & (data['Speed'] > 0) & (data['TravelDistance'] > 0) & (data['TravelTime'] > 0)]

# 保存清洗后的数据 1分
```

### 第一种方法

```
traffic_event_counts = data['TrafficEvent'].value_counts()
print("每种交通事件的发生次数:\n", traffic_event_counts)

# 统计不同性别的平均车速、行驶距离和行驶时间 2分
gender_stats = data.groupby('Gender')[['Speed', 'TravelDistance', 'TravelTime']].mean()
print("不同性别的平均车速、行驶距离和行驶时间:\n", gender_stats)

# 统计不同年龄段的驾驶员数 5分
```

### 第二种方法

- 通过字典，将聚合字段，聚合方式一一对应，

```
# 统计不同性别的平均车速、行驶距离和行驶时间 2分
gender_stats = data.groupby('Gender').agg({'Speed': 'mean', 'TravelDistance': 'mean', 'TravelTime': 'mean'})
print("不同性别的平均车速、行驶距离和行驶时间:\n", gender_stats)
```